

K L UNIVERSITY
FRESHMAN ENGINEERING DEPARTMENT

A Project Based Lab Report

On

Lost Item Logger System for College Campus (LIL-CAMP)

SUBMITTED BY:

ID NUMBER	NAME
2500040219	N. Sriram Goud
2500040220	Voonna Nikesh
2500040260	Dhanalakshmi
2500040135	Harsha Vardhan

UNDER THE ESTEEMED GUIDANCE OF

Dr. . Alangudi Balaji

PROFESSOR



KL UNIVERSITY
Green fields, Vaddeswaram – 522 502
Guntur Dt., AP, India.

DEPARTMENT OF BASIC ENGINEERING SCIENCES



CERTIFICATE

This is to certify that the project based laboratory report entitled “**Lost Item Logger System for College Campus (LIL-CAMP)**” submitted by N. Sriram Goud, Nikesh , Dhanalakshmi, Harsha Vardhan bearing Regd. No. 2500040219, 2500040220, 2500040260, 2500040135 to the **Department of ELECTRONIC COMMUNICATION ENGINEERING , KL University** in partial fulfillment of the requirements for the completion of a project based Laboratory in “Python Programming (25SC2007)” course in I B Tech II Semester, is a Bonafede record of the work carried out by him/her under my supervision during the academic year 2025 – 2026.

PROJECT SUPERVISOR

Dr. N. Alangudi Balaji

HEAD OF THE DEPARTMENT

Dr. D. HARITHA

ACKNOWLEDGEMENTS

It is great pleasure for me to express my gratitude to our honorable President.

Sri. Koneru Satyanarayana, for giving the opportunity and platform with facilities in accomplishing the project-based laboratory report.

I express sincere gratitude to our Director **Dr. A. Jagdeesh** for his administration towards our academic growth.

I express sincere gratitude to our Coordinator and HOD-BES I, **Dr. D. Haritha** for her leadership and constant motivation provided in successful completion of our academic semester. I record it as my privilege to deeply thank you for providing us with the efficient faculty and facilities to make our ideas into reality.

I express my sincere thanks to our project supervisor **Dr. Alangudi Balaji** for her novel association of ideas, encouragement, appreciation, and intellectual zeal which motivated us to venture this project successfully.

Finally, we are pleased to acknowledge our indebtedness to all those who devoted themselves directly or indirectly to making this project a success.

ID NUMBER	NAME
2500040219	N. Sriram Goud
2500040220	Voonna Nikesh
2500040260	Dhanalakshmi
2500040135	Harsha Vardhan

ABSTRACT

The **Lost Item Logger System for College Campus (LIL-CAMP)** is a Python-based software solution developed to simplify the process of reporting, tracking, and recovering lost and found items within a college campus. In large educational institutions, students and staff frequently misplace important belongings such as ID cards, books, wallets, calculators, USB drives, keys, and mobile phones. Traditional manual methods such as notice boards, word of mouth, or security desk complaints are often inefficient, time-consuming, and unreliable.

This project provides a centralized digital platform where users can:

- Register lost items with complete details
- Report found items instantly
- Search existing records by item name or category
- Match similar lost and found items automatically
- Store records securely using JSON files
- Maintain a transparent and organized recovery process

The system is designed using various Python programming concepts learned during the semester, including:

- Variables and data types
- Conditional statements and loops
- Functions and modular programming
- Lists, dictionaries, and sets
- Object Oriented Programming concepts
- File handling using JSON
- Exception handling for validation

This project improves campus communication, reduces manual effort, saves time, and significantly increases the chances of item recovery. It also helps in maintaining a digital history of lost and found records for future reference.

Keywords: lost item logger, college campus, python project, lost and found system, digital tracking.

INDEX

S.NO	TITLE	PAGE NO
1	Introduction	6
2	Aim of the Project	6
2.1	Advantages & Disadvantages	7
2.2	Future Implementation	7
3	Software & Hardware Details	8
4	Data Flow Diagram	9
5	Algorithm for each module	10
6	Implementation	11-14
7	Integration and System Testing	15-17
8	Conclusion	18

INTRODUCTION

College campuses are busy places where students often lose important items such as ID cards, wallets, books, calculators, and mobile phones.

In many colleges, there is no proper system to manage lost and found items. Students usually depend on friends or campus staff, which causes delay and confusion.

The **Lost Item Logger System for College Campus (LIL-CAMP)** is developed to solve this problem. It helps users:

- Register lost items
- Report found items
- Search item records
- Match similar items
- Contact the owner or finder

This system reduces manual work and makes item recovery faster and easier.

AIM

The primary aim of the Lost Item Logger System for College Campus is to create an efficient and user-friendly digital solution for managing lost and found items within the college premises.

In many campuses, students lose important items regularly, and the absence of a proper system causes inconvenience and stress. This project is designed to solve that issue by maintaining an organized digital record of all lost and found items.

Objectives:

- To provide a simple interface for reporting lost items
- To create a platform for reporting found items
- To maintain item details in a secure digital format
- To reduce time and effort in locating lost belongings
- To improve communication between finder and owner
- To increase the chances of successful item recovery
- To reduce manual paperwork and confusion
- To build a real-time searchable database for campus use

ADVANTAGES

The Lost Item Logger System provides several practical benefits:

- Easy and simple interface for users
- Saves time in reporting and searching lost items
- Reduces manual paperwork and confusion
- Maintains proper digital records for future reference
- Helps in faster recovery of misplaced belongings
- Allows easy search and filtering of items
- Improves campus communication and transparency
- Useful for both students and staff
- Can be upgraded in future with notifications and mobile app support

DISADVANTAGES

Although useful, the system has some limitations:

- Requires access to a computer or digital device
- Depends on users entering correct details
- Needs regular maintenance of records
- Limited to campus users unless expanded
- Initial awareness is required for effective use

FUTURE IMPLEMENTATION

The current version of the project is developed as a basic Python application. In future, the system can be enhanced with advanced features such as:

- SMS alerts to notify users when similar items are found
- Email notifications for status updates
- Mobile application support for Android and iOS
- QR code based item tracking
- Cloud database storage for better accessibility
- AI-based smart matching system using keywords
- Image upload support for better identification

SYSTEM REQUIREMENTS

➤ SOFTWARE REQUIREMENTS:

- The major software requirements of the project are as follows:

Language: Turbo-C.

- Operating system: Windows 10 or more.
- Technical requirements: Monitor, CPU, Keyboard, Mouse, etc...
- The software tools used in this project are:
- Python 3.x programming language
- VS Code / PyCharm / IDLE for coding
- JSON library for file handling
- Operating System: Windows 10 / 11 or Linux
- Optional libraries for future upgrades:
- pandas
- matplotlib

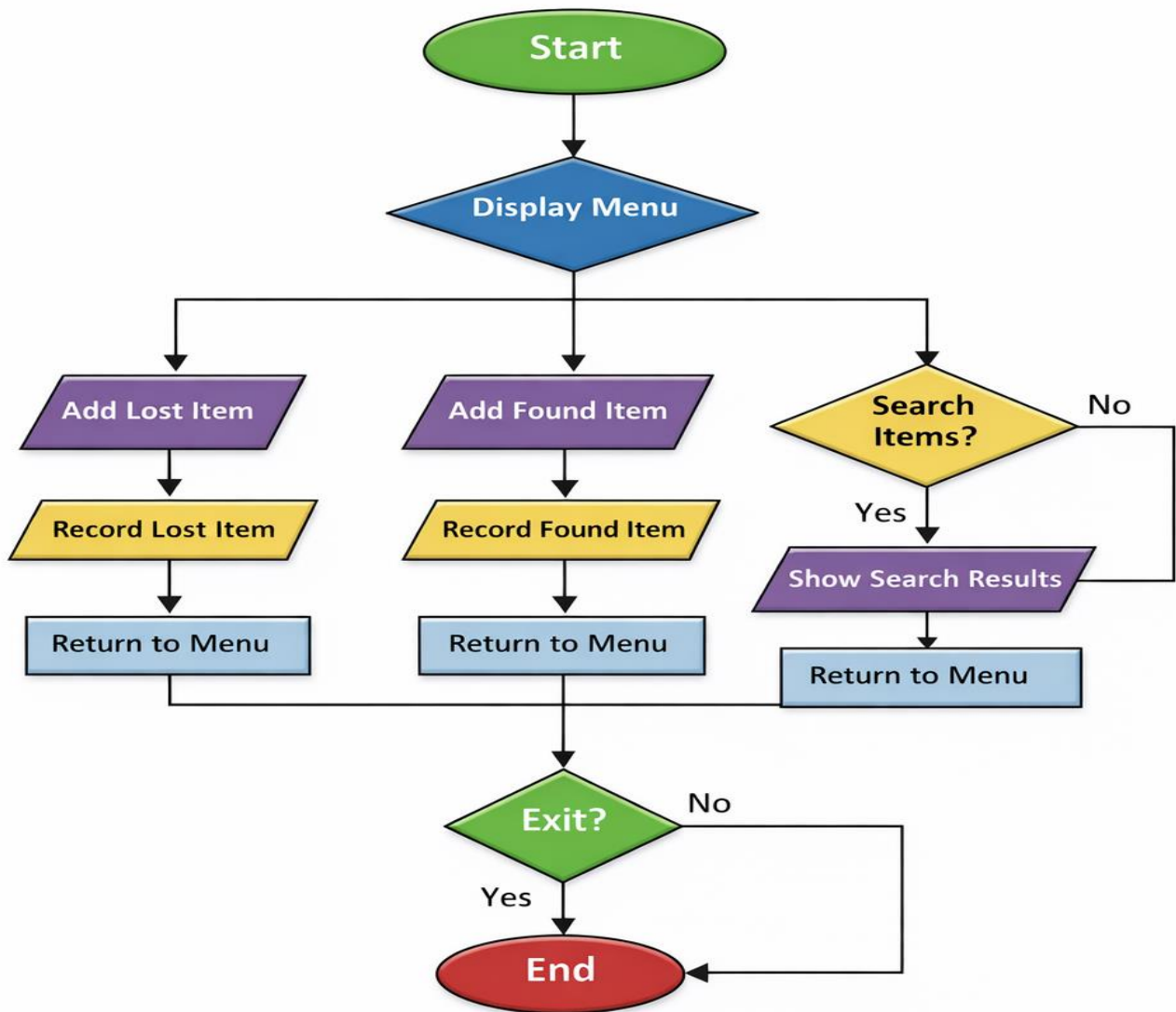
➤ HARDWARE REQUIREMENTS:

The minimum hardware required for smooth execution:

- Laptop / Desktop Computer
- Minimum 4GB RAM
- Intel i3 / Ryzen 3 processor or above
- Keyboard and mouse
- Internet connection (optional for future online version)
- Minimum 500 MB free storage

DATA FLOW DIAGRAM

Lost Item Logger System



ALGORITHM

Step 1: Start the program.

Step 2: Display main menu.

Step 3: Choose:

- Add Lost Item
- Add Found Item
- Search Item
- View Items

Step 4: Accept item details.

Step 5: Validate input.

Step 6: Store data in JSON file.

Step 7: Search and match similar items.

Step 8: Display results.

Step 9: Exit program.

IMPLEMENTATION

```
import json
import os
from datetime import datetime

# Automatically locate your computer's user "Home" folder to bypass permission errors
HOME_DIR = os.path.expanduser("~")
LOST_FILE = os.path.join(HOME_DIR, "lost_items.json")
FOUND_FILE = os.path.join(HOME_DIR, "found_items.json")

class Item:
    def __init__(self, name, description, location, date, contact, category, status):
        self.name = name
        self.description = description
        self.location = location
        self.date = date
        self.contact = contact
        self.category = category
        self.status = status

    def to_dict(self):
        return {
            "name": self.name,
            "description": self.description,
            "location": self.location,
            "date": self.date,
            "contact": self.contact,
            "category": self.category,
            "status": self.status,
        }

    def display(self):
        print(f"\nName: {self.name}")
        print(f>Description: {self.description}")
        print(f>Location: {self.location}")
        print(f>Date: {self.date}")
        print(f>Contact: {self.contact}")
        print(f>Category: {self.category}")
        print(f>Status: {self.status}")

class LostItemLogger:
    def __init__(self):
        self.lost_items = self.load_items(LOST_FILE)
        self.found_items = self.load_items(FOUND_FILE)

    def load_items(self, filename):
        try:
```

```

        with open(filename, "r") as file:
            return json.load(file)
    except (FileNotFoundError, json.JSONDecodeError):
        return []

def save_items(self, filename, items):
    try:
        with open(filename, "w") as file:
            json.dump(items, file, indent=4)
        return True # Tell the system it saved successfully
    except PermissionError:
        print(f"\n[ERROR] Your system blocked saving to: {filename}")
        return False
    except Exception as e:
        print(f"\n[ERROR] An unexpected error occurred: {e}")
        return False

def validate_date(self, date_text):
    try:
        datetime.strptime(date_text, "%d-%m-%Y")
        return True
    except ValueError:
        return False

def add_item(self, status):
    print(f"\nEnter {status} item details")
    name = input("Item Name: ").strip()
    description = input("Description: ").strip()
    location = input("Location: ").strip()
    date = input("Date (DD-MM-YYYY): ").strip()

    if not self.validate_date(date):
        print("Invalid date format. Use DD-MM-YYYY.")
        return

    contact = input("Contact: ").strip()
    category = input("Category: ").strip()

    item = Item(name, description, location, date, contact, category, status)
    item_data = item.to_dict()

    # Try to save and store the result (True or False)
    if status == "Lost":
        self.lost_items.append(item_data)
        saved_successfully = self.save_items(LOST_FILE, self.lost_items)
        file_path = LOST_FILE
    else:
        self.found_items.append(item_data)
        saved_successfully = self.save_items(FOUND_FILE, self.found_items)
        file_path = FOUND_FILE

```

```

# Give accurate feedback based on whether it saved
if saved_successfully:
    print(f"\n✅ {status} item added and saved successfully!")
    print(f"📁 Saved file location: {file_path}")
else:
    print(f"\n⚠️ {status} item added to memory, but COULD NOT BE SAVED to your hard drive.")

self.match_items(item_data)

def list_items(self, status):
    items = self.lost_items if status == "Lost" else self.found_items
    if not items:
        print(f"No {status.lower()} items found.")
        return

    print(f"\n--- {status} Items ---")
    for i, item in enumerate(items, 1):
        print(f"\nRecord {i}")
        for key, value in item.items():
            print(f"{key.capitalize():} {value}")

def search_item(self):
    keyword = input("Enter item name to search: ").strip().lower()
    found = False

    print("\nSearch Results:")
    for collection in [self.lost_items, self.found_items]:
        for item in collection:
            if keyword in item["name"].lower():
                found = True
                for key, value in item.items():
                    print(f"{key.capitalize():} {value}")
                print("-" * 30)

    if not found:
        print("No matching items found.")

def match_items(self, new_item):
    target_list = self.found_items if new_item["status"] == "Lost" else self.lost_items

    for item in target_list:
        if (
            new_item["name"].lower() == item["name"].lower()
            or new_item["category"].lower() == item["category"].lower()
        ):
            print("\nPossible Match Found!")
            print("Existing Record:")
            for key, value in item.items():
                print(f"{key.capitalize():} {value}")

```

```

        print("Contact the person to verify.")
        return

def show_menu(self):
    while True:
        print("\n===== LOST ITEM LOGGER SYSTEM =====")
        print("1. Add Lost Item")
        print("2. Add Found Item")
        print("3. Search Item")
        print("4. List Lost Items")
        print("5. List Found Items")
        print("6. Exit")

        try:
            choice = int(input("Enter your choice: "))
        except ValueError:
            print("Please enter a valid number.")
            continue

        if choice == 1:
            self.add_item("Lost")
        elif choice == 2:
            self.add_item("Found")
        elif choice == 3:
            self.search_item()
        elif choice == 4:
            self.list_items("Lost")
        elif choice == 5:
            self.list_items("Found")
        elif choice == 6:
            print("Exiting system...")
            break
        else:
            print("Invalid choice. Try again.")

if __name__ == "__main__":
    system = LostItemLogger()
    system.show_menu()

```

INTEGRATION AND SYSTEM TESTING OUTPUTS

#	Test Case No.	Module Tested	Input Given	Expected Output	Actual Output	Status
	1	Main Menu	Select option 1	Add Lost Item screen opens	Add Lost Item screen displayed	Pass
	2	Add Lost Item	Enter item details	Lost item stored successfully	Lost item stored	Pass
	3	Add Found Item	Enter found item details	Found item stored successfully	Found item stored	Pass
	4	Search Item	Search by item name	Matching item displayed	Matching item shown	Pass
	5	Item Matching	Similar lost and found items	Match alert shown	Match detected	Pass
	6	File Handling	Save data	Data stored in JSON file	File saved successfully	Pass
	7	Input Validation	Wrong date format	Error message displayed	Validation worked	Pass
	8	Exit Option	Select Exit	Program terminates	Program closed	Pass

Screen Shots:

```

===== LOST ITEM LOGGER SYSTEM =====
1. Add Lost Item
2. Add Found Item
3. Search Item
4. List Lost Items
5. List Found Items
6. Exit
Enter your choice: > 1

Enter Lost item details
Item Name: > Blue Backpack
Description: > Jansport bag with a laptop inside
Location: > Main Library
Date (DD-MM-YYYY): > 10-04-2026
Contact: > john.doe@email.com
Category: > Bags
Lost item added successfully.

```

===== LOST ITEM LOGGER SYSTEM =====

1. Add Lost Item
2. Add Found Item
3. Search Item
4. List Lost Items
5. List Found Items
6. Exit

Enter your choice: > 2

Enter Found item details

Item Name: > Blue Backpack

Description: > Found near the computers

Location: > Main Library 2nd Floor

Date (DD-MM-YYYY): > 10-04-2026

Contact: > library_desk@email.com

Category: > Bags

Found item added successfully.

Possible Match Found!

Existing Record:

Name: Blue Backpack

Description: Jansport bag with a laptop inside

Location: Main Library

Date: 10-04-2026

Contact: john.doe@email.com

Category: Bags

Status: Lost

Contact the person to verify.

===== LOST ITEM LOGGER SYSTEM =====

1. Add Lost Item
2. Add Found Item
3. Search Item
4. List Lost Items
5. List Found Items
6. Exit

Enter your choice: > 3

Enter item name to search: > backpack


```
Search Results:
Name: Blue Backpack
Description: Jansport bag with a laptop inside
Location: Main Library
Date: 10-04-2026
Contact: john.doe@email.com
Category: Bags
Status: Lost
```

```
-----
Name: Blue Backpack
Description: Found near the computers
Location: Main Library 2nd Floor
Date: 10-04-2026
Contact: library_desk@email.com
Category: Bags
Status: Found
```

```
===== LOST ITEM LOGGER SYSTEM =====
```

1. Add Lost Item
2. Add Found Item
3. Search Item
4. List Lost Items
5. List Found Items
6. Exit

```
Enter your choice: > 6
```

```
Exiting system...
```

CONCLUSION

The Lost Item Logger System for College Campus (LIL-CAMP) was successfully designed and implemented using Python programming concepts. The project fulfills its main objective of providing a simple, effective, and organized system to manage lost and found items in a college campus.

Through this project, users can:

- Register lost items easily
- Report found items quickly
- Search item details efficiently
- Maintain digital records permanently
- Improve communication between finder and owner

This project demonstrates the practical application of Python concepts such as input/output, lists, dictionaries, functions, classes, file handling, and exception handling. It also provides students with hands-on experience in solving real-world problems using programming.

Overall, the system is useful, reliable, and highly beneficial for college campuses. With future enhancements, it can become a complete smart campus lost and found management system.